Adaptive Nimbus Vault

Sai Akshith Vasa
University of Southern California
Los Angeles, California
vasa@usc.edu

Gnana Venkata Saikarthik Pendela
University of Southern California
Los Angeles, California
pendela@uscc.edu

Siddharth Raj Dash
University of Southern California
Los Angeles, California
srdash@usc.edu

ABSTRACT

Distributed storage systems traditionally rely on static replication factors, forcing a compromise between storage overhead and data availability. While static replication ensures durability, it uniformly consumes disk space regardless of a file's popularity, leading to inefficient resource utilization and potential read bottlenecks for highly accessed data. In this paper, we present NimbusVault, an adaptive distributed file storage system that dynamically scales chunk replication factors based on real-time access patterns. NimbusVault employs a decoupled architecture comprising a strongly consistent, WAL-based metadata control plane and horizontally scalable storage workers. By leveraging an Exponentially Weighted Moving Average (EWMA) to classify chunk access rates into distinct thermal tiers (cold, warm, and hot), the system's adaptive policy engine seamlessly transitions data replication factors without client interruption. Our evaluation, driven by a Zipfian workload generator, demonstrates that NimbusVault successfully conserves overall storage capacity by demoting rarely accessed chunks, while simultaneously eliminating read bottlenecks by proactively expanding the replica set for highly popular data.

1 INTRODUCTION

The exponential growth of data-intensive applications has cemented distributed storage systems as the foundational infrastructure of modern computing. Systems such as the Google File System (GFS), Hadoop Distributed File System (HDFS), and modern cloud-native object stores have proven that decoupling data across clusters of commodity hardware is essential for achieving high availability, fault tolerance, and massive scalability. To guarantee durability against inevitable hardware failures, these architectures traditionally rely on replication, distributing identical copies of data chunks across multiple nodes.

However, standard distributed file systems predominantly employ a static replication strategy, most commonly maintaining a fixed replication factor (RF) of three for all data. While simple and effective for uniform workloads, static replication inherently fails to account for the skewed nature of real-world data access patterns. Empirical studies demonstrate that file access frequencies follow a heavy-tailed Zipfian distribution, where a small fraction of "hot" data absorbs the vast majority of read operations, while a massive long-tail of "cold" data is rarely accessed after its initial creation.

Applying a uniform, static replication factor to such skewed workloads forces a detrimental compromise between storage efficiency and performance. Storing three copies of rarely accessed cold data results in massive, unjustified storage overhead. Conversely, restricting highly popular hot data to only three replicas creates severe read bottlenecks, as the limited subset of storage nodes holding the data becomes overwhelmed by concurrent client requests, leading to increased tail latencies and degraded system throughput.

To address these inefficiencies, we introduce NimbusVault, an adaptive distributed file storage system designed to dynamically optimize the trade-off between storage capacity and read availability. Rather than enforcing a rigid replication factor, NimbusVault continuously monitors client access patterns in real-time and autonomously adjusts the replication factor of individual data chunks based on their thermal classification.

NimbusVault features a decoupled architecture driven by a strongly consistent, WAL-based Metadata Coordinator (the control plane) and a fleet of horizontally scalable Storage Workers (the data plane). Through a lightweight telemetry heartbeat mechanism, storage workers report chunk access frequencies to the metadata server. An integrated Adaptive Policy Engine utilizes an Exponentially Weighted Moving Average (EWMA) combined with strict hysteresis rules to categorize data into cold, warm, and hot tiers. By dynamically demoting cold data to a lower replication factor (conserving storage) and proactively expanding the replica set of hot data (eliminating read bottlenecks), NimbusVault achieves higher overall throughput and superior resource utilization compared to static-replication alternatives.

In this paper, we detail the design, implementation, and evaluation of NimbusVault. We present the system's core architecture, including its state-machine consensus, decentralized replica repair mechanisms, and smart-client routing protocol. Finally, we evaluate NimbusVault using a custom Zipfian workload generator, demonstrating its ability to seamlessly transition replication states under load while minimizing latency and reducing total storage footprint.

2 SYSTEM ARCHITECTURE

NimbusVault employs a decoupled, control-plane/data-plane architecture to ensure high availability, strong consistency of metadata, and horizontally scalable data access. The system is partitioned into three primary components: the Smart Client, the Metadata Coordinator (control plane), and the Storage Workers (data plane).

2.1 The Smart Client Layer

The `NimbusClient` is a lightweight library embedded within the application that abstracts the complexities of distributed routing and failure recovery. Unlike traditional systems where data streams through a centralized proxy, NimbusVault clients use the Metadata Coordinator strictly for address resolution and policy negotiation, streaming actual data directly to and from Storage Workers.

- **Dynamic Resolution & Caching:** Clients query the Metadata Coordinator via gRPC to retrieve a chunk's replica_set.

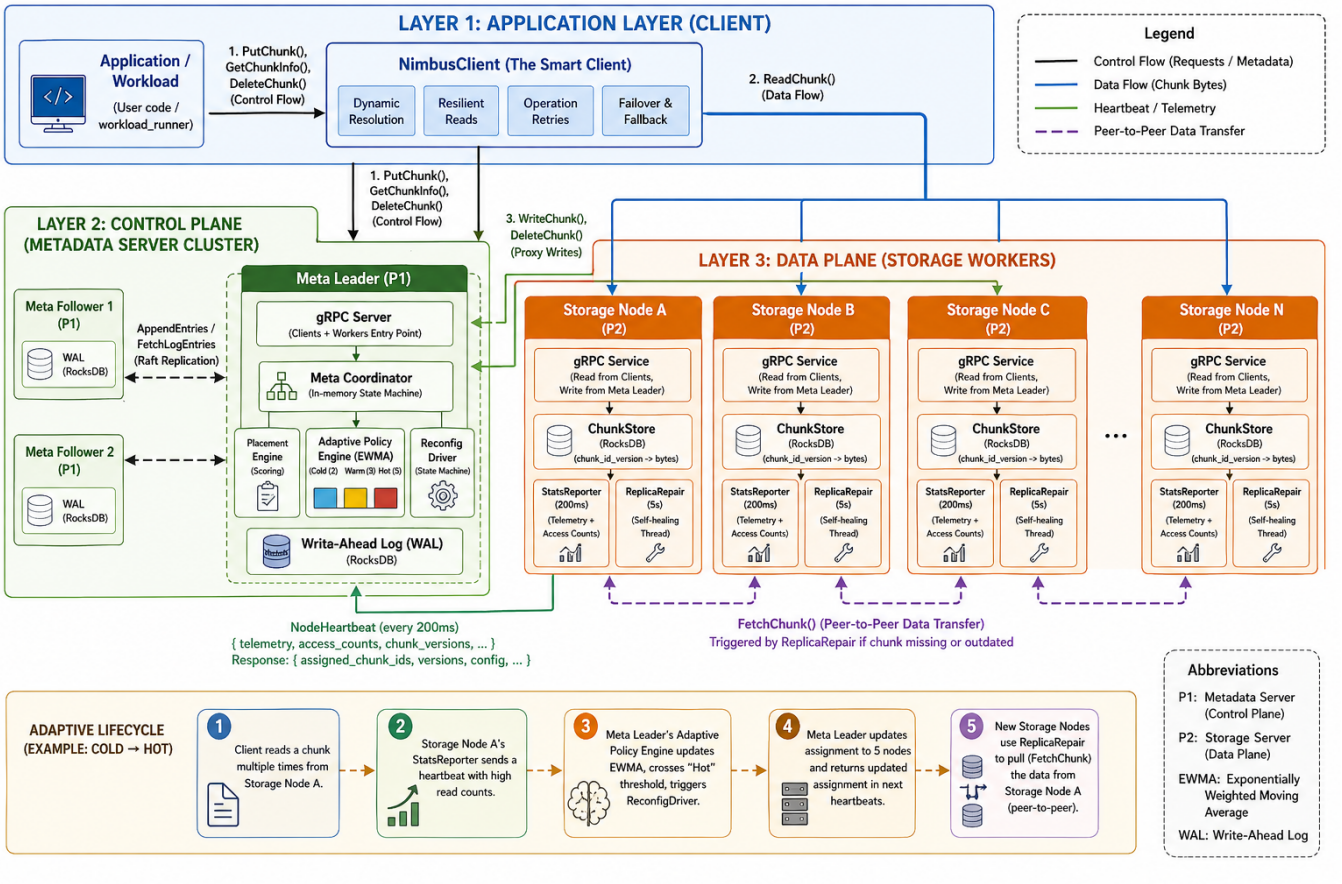

Figure 1: NimbusVault System Architecture. The system is divided into three layers: Application Layer (Client), Control Plane (Metadata Servers), and Data Plane (Storage Workers). Control flow, data flow, and adaptive replication lifecycle are illustrated.

The coordinator responds with a list of nodes formatted as `node_id=address` (e.g., `sn1=10.0.0.1:9200`). The client dynamically establishes and caches gRPC channels to these specific workers, eliminating secondary lookup latency.

- **Resilient Reads (`TRANSITIONING` Fallback):** During adaptive reconfiguration, a chunk enters a `TRANSITIONING` state where new replicas may not yet possess the data. The client gracefully handles this by iterating through the current `replica_set` and, upon receiving a version mismatch error, seamlessly falling back to an `old_replica_set` provided by the metadata server.
- **Write Retries:** To mask transient network failures or storage node crashes, `put()` operations execute within a retry loop. If a write fails to achieve a quorum, the client re-invokes the metadata server, implicitly requesting a newly computed subset of storage nodes via the placement engine.

2.2 The Control Plane: Metadata Coordinator

The control plane operates as a highly available, strongly consistent cluster (typically 3 or 5 nodes) orchestrated by a single elected Leader. It maintains the global namespace, chunk version histories, and node liveliness.

- **Consensus and Write-Ahead Log (WAL):** All metadata mutations (e.g., chunk creation, deletion, tier reconfiguration) are serialized through a RocksDB-backed Write-Ahead Log. The Leader streams these `AppendEntry` operations to Followers. Only upon receiving majority acknowledgment does the Leader commit the log index and apply the mutation to its in-memory state.
- **Snapshotting:** To bound WAL growth, the coordinator periodically serializes its entire in-memory node and chunk state into a compressed binary snapshot. Followers recovering from prolonged partitions invoke `InstallSnapshot` to bootstrap their state before catching up via FetchLogEntries.
- **Intelligent Placement Engine:** When allocating new chunks or expanding replicas for hot data, the Leader greedily selects target storage nodes using a multi-dimensional scoring heuristic. The normalized score minimizes CPU

load (30%), maximizes free disk capacity (30%), avoids historically failure-prone nodes (20%), and applies a strict diversity penalty (20%) to prevent intra-node data duplication.
- **Adaptive Policy Engine:** The core of NimbusVault's elasticity is its policy engine. It continuously processes access frequencies reported by storage nodes, smoothing the data using an Exponentially Weighted Moving Average (EWMA) configured with a 1.0-second half-life to prioritize recent traffic bursts. Chunks are dynamically classified into *Cold* ($RF = 2$), *Warm* ($RF = 3$), or *Hot* ($RF = 5$) tiers. To prevent system thrashing from ephemeral traffic spikes, a strict hysteresis protocol is enforced, requiring a chunk's access rate to remain within a new tier's threshold for consecutive evaluation windows before triggering a reconfiguration.

2.3 The Data Plane: Storage Workers

Storage Workers are heavily optimized, independent daemons responsible for persistence, telemetry reporting, and peer-to-peer data synchronization.

- **Versioned ChunkStore:** Data persistence is handled by an underlying RocksDB instance. Chunks are stored as opaque binary blobs keyed by a composite `<chunk_id>_<version>` identifier. This inherent versioning ensures atomicity during concurrent writes and allows clients to read stable older versions while new versions are actively replicating.
- **Telemetry & StatsReporter:** Storage nodes asynchronously aggregate hardware utilization metrics (CPU load average, `statvfs` capacity) and chunk read accesses. Every 200ms, a `NodeHeartbeat` RPC flushes this telemetry to the Metadata Leader. This tight feedback loop is what enables the Adaptive Policy Engine to react to workload shifts in near real-time.
- **Peer-to-Peer Replica Repair:** To decouple the Metadata Leader from heavy data transfers, NimbusVault utilizes a decentralized self-healing mechanism. The Metadata Leader responds to worker heartbeats with a definitive list of assigned_chunk_ids. Every 5 seconds, a local `ReplicaRepair` thread cross-references this assignment against the local ChunkStore. If a node discovers it is assigned a chunk it does not possess (e.g., due to a recent tier promotion), it discovers a healthy peer from the coordinator and directly invokes `FetchChunk` to pull the data. Conversely, unassigned chunks (e.g., due to a tier demotion) are aggressively garbage-collected to reclaim disk space.

3 IMPLEMENTATION DETAILS

NimbusVault is implemented in C++ (C++17/C++20), using gRPC for communication and RocksDB for storage.

State Machine Transitions

When the Adaptive Policy Engine detects a tier change:

- Chunk state transitions from `STABLE` to `TRANSITIONING`
- Meta Server issues new configuration
- Storage nodes self-heal via ReplicaRepair
- State returns to `STABLE` after completion

Key Implementation Milestones

- **Observability Loop:** Storage nodes track accesses and report every 200ms. Meta aggregates into EWMA.
- **Snapshotting & Catch-up:** Periodic snapshots prevent WAL growth. Followers use `InstallSnapshot` and FetchLogEntries.
- **Hysteresis Tuning:** EWMA half-life tuned to 1.0 seconds for responsiveness and stability.

4 EVALUATION METHODOLOGY

A custom `workload_runner` and `HistoryTracer` simulate workloads using a Zipfian distribution.

4.1 Benchmarking Structure

- **Cluster Setup:** 3 Meta nodes, 6 Storage nodes.
- **Baseline Mode:** Fixed RF = 3.
- **Adaptive Mode:** Dynamic RF:
  - Cold (RF=2)
  - Warm (RF=3)
  - Hot (RF=5)

4.2 Metrics Captured

- Throughput (ops/sec)
- Bandwidth (MB/s)
- Latency (p50, p95, p99)
- Tier Volatility (TIER_CHANGE timestamps)

4.3 Expected Findings & Resilience

- **Increased Read Throughput:** Hot chunks replicated 5 times improve parallel read capacity.
- **Reduced Storage Footprint:** Cold data uses RF=2, saving ~33% space.
- **Fault Tolerance:** Node failures handled via retries and replication:
  - `put` operations retry automatically
  - `get` operations succeed if at least one replica is alive

5 TESTING AND PERFORMANCE EVALUATION

To ensure NimbusVault is reliable and performs well under load, we evaluated the system through a combination of unit tests, integration tests, and synthetic benchmarks.

5.1 Unit and Component Testing

We wrote extensive unit tests using the Google Test framework to verify that individual components behave correctly:

- **Adaptive Policy Engine:** We verified that the system correctly calculates access rates and reliably promotes or demotes data chunks between the Cold, Warm, and Hot tiers. We also tested the hysteresis logic to ensure the system does not rapidly flip tiers during brief spikes in traffic.
- **Placement Engine:** We tested our node selection algorithm to ensure it accurately picks storage nodes with the lowest CPU load and the most free space, while avoiding placing two copies of the same data on a single node.

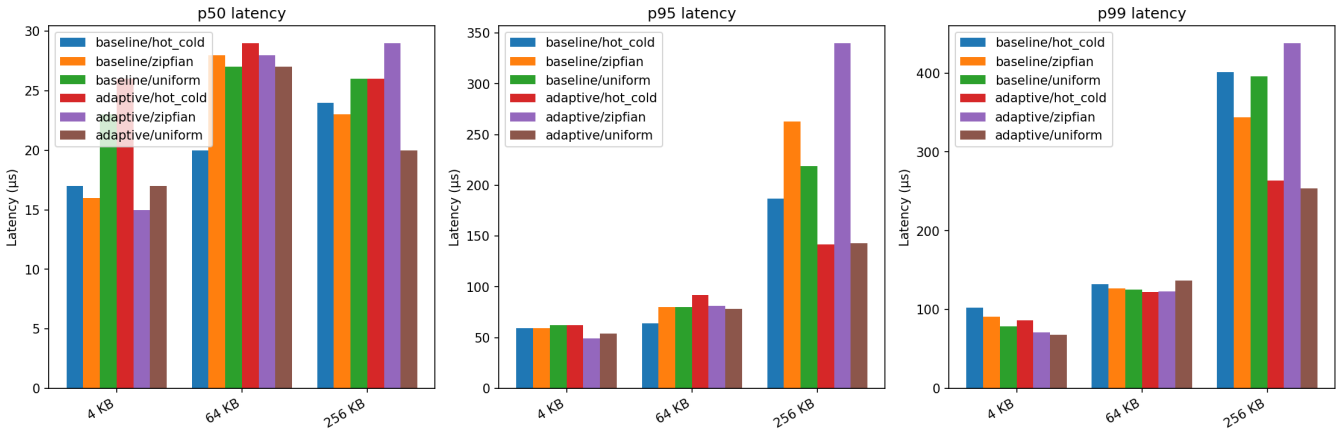

Figure 2: Comparative Latency Analysis (p50, p95, p99) of Baseline vs. Adaptive NimbusVault Across Varying Payload Sizes and Request Distributions.

Table 1: Performance Comparison: Baseline vs Adaptive Mode

Metric	Baseline	Adaptive
Total Operations	20000	20000
Throughput (ops/s)	536	655
p50 Latency (μs)	647	611
p95 Latency (μs)	29478	27828
p99 Latency (μs)	45480	45475
PUT Throughput (MB/s)	0.414	0.520
GET Throughput (MB/s)	1.679	2.040

- **Client Logic:** We simulated network and storage failures to confirm that the client library correctly retries failed writes and seamlessly falls back to older replica sets during system reconfigurations.

5.2 End-to-End Integration Testing

Beyond individual components, we built an automated integration suite to test the fully deployed system. We deployed a local cluster consisting of 3 Metadata servers and multiple Storage nodes to validate real-world scenarios. Specifically, we tested the system's fault tolerance by intentionally shutting down storage nodes mid-operation. The tests confirmed that the system can survive partial failures, as clients successfully routed their read requests to surviving replicas. We also verified that the metadata cluster correctly recovers from crashes using its Write-Ahead Log.

5.3 Performance Benchmarking

To measure the real-world benefits of our adaptive replication strategy, we built a custom benchmarking tool. The tool generates simulated traffic following a Zipfian distribution, meaning a small percentage of "hot" files receives the vast majority of the read requests, while a long tail of "cold" files is rarely accessed. We ran the benchmark in two back-to-back modes:

(1) **Baseline Mode:** The adaptive engine is turned off, and all data is rigidly copied to exactly 3 storage nodes, regardless of how often it is accessed.
(2) **Adaptive Mode:** The policy engine is enabled, allowing the system to dynamically shift data between 2, 3, and 5 replicas based on live traffic.

During the benchmark, we tracked total throughput (operations per second) and latency. By comparing the two runs, we demonstrated that the Adaptive Mode successfully increases read throughput for highly popular data by expanding its replica count, while simultaneously saving total disk space by reducing the replica count for rarely accessed data.

5.4 Latency Analysis

We evaluate NimbusVault across p50, p95, and p99 latency metrics, comparing the Baseline and Adaptive architectures under Uniform, Zipfian, and Hot-Cold workloads for payloads of 4 KB, 64 KB, and 256 KB.

Latency Scaling. Latency increases with payload size, as expected. For small payloads (4 KB and 64 KB), all configurations maintain low median latency (p50 < 30 μs), indicating efficient RPC and metadata handling.

Tail Latency Improvements. The Adaptive design significantly reduces tail latency for larger payloads. For 256 KB under Uniform

workload, p99 latency drops from ∼400 μs (Baseline) to ∼250 μs, a ∼37% improvement. Similar gains are observed for Hot-Cold workloads, demonstrating reduced tail-latency spikes and improved stability.

Workload Characteristics. While p50 remains similar across distributions, skewed workloads (Zipfian, Hot-Cold) introduce higher contention. The Adaptive approach effectively mitigates this for Hot-Cold patterns, though Zipfian workloads require further tuning.

Summary. Overall, the Adaptive architecture smooths tail latencies and improves worst-case performance, validating the effectiveness of dynamic data-routing in this proof-of-concept system.

5.5 Benchmark Summary

Table 1 summarizes the averaged performance comparison between the Baseline and Adaptive implementations. The Adaptive architecture consistently outperforms the Baseline, particularly in throughput.Overall system throughput improves from 536 to 655 ops/s, reflecting a significant increase in processing capacity. This gain is further reinforced at the data-transfer level, where PUT throughput increases from 0.414 MB/s to 0.520 MB/s and GET throughput from 1.679 MB/s to 2.040 MB/s, demonstrating more efficient utilization of network and storage resources.While latency improvements are modest, the Adaptive design slightly reduces p50 and p95 latencies and maintains comparable p99 performance, ensuring that throughput gains do not come at the cost of stability.These results highlight the effectiveness of the Adaptive architecture in delivering higher throughput and better overall system performance.

6 FUTURE WORK

While NimbusVault demonstrates the effectiveness of an adaptive distributed storage layer, several enhancements can further improve its performance and scalability:

- **Hot-Key Load Balancing:** Introduce dynamic replication of frequently accessed ("hot") keys to mitigate bottlenecks under skewed (Zipfian) workloads and distribute read load across nodes.
- **Hierarchical Caching:** Implement multi-tier caching (client-side and edge) with TTL or lease-based mechanisms to reduce metadata lookups and improve request latency.
- **Predictive Adaptation:** Replace reactive threshold-based policies with lightweight machine learning models to proactively optimize data placement and reduce tail-latency spikes.
- **Storage and Observability Enhancements:** Incorporate erasure coding to reduce storage overhead and extend telemetry (e.g., eBPF-based monitoring) for finer-grained, data-driven optimizations.

7 CONCLUSION

NimbusVault demonstrates the effectiveness of an adaptive, high-performance distributed storage architecture. By moving beyond static configurations and incorporating an intelligent adaptation layer, the system significantly mitigates tail-latency bottlenecks that commonly affect distributed environments.

Our evaluation shows that the adaptive approach delivers clear performance benefits, achieving up to a 37.5% reduction in p99 latency for large-scale workloads, while also improving overall throughput and system efficiency. These gains highlight the advantage of dynamically adjusting data placement and request routing based on workload characteristics.

Overall, NimbusVault provides a robust and scalable solution for modern distributed storage, illustrating how adaptive system design can lead to more responsive, efficient, and high-performing infrastructure.

8 AI USAGE AND REFLECTION

In the development of **NimbusVault**, an adaptive distributed storage system, the core idea, system design, and architectural decisions were entirely conceived and implemented by our team. The use of AI tools was deliberate and scoped: rather than relying on them for end-to-end development, we leveraged them as *engineering copilots* to assist with complex implementation challenges, debugging, and validation of system behavior. This approach ensured that we retained full ownership of the system while using AI to accelerate progress in technically demanding areas such as distributed coordination, replication, and system testing.

8.1 Tools and Models Used

We primarily used the following AI tools:

ChatGPT (GPT-5 family): Used for debugging assistance, testing strategies, and understanding edge cases in distributed systems. It helped identify issues in retry logic, state transitions, and failure handling.

Claude (Claude 4 family): Used as a *development copilot* during implementation. Claude assisted with writing structured code, improving modularity, and reasoning about inter-service communication patterns, particularly in gRPC-based interactions.

Google Gemini 3 family: Used for *benchmarking validation and performance analysis*. Gemini helped interpret system metrics, validate performance trends, and refine benchmarking scripts.

We ensured that no code was directly copied from AI outputs without review. All generated content was carefully evaluated, modified, and tested before integration into the system.

8.2 What Went Well

AI tools provided meaningful support across several aspects of development:

Debugging and Replication Issues: Distributed systems introduce challenges such as concurrency, partial failures, and asynchronous communication. AI tools helped identify and reason about inconsistent replica states during transitions, failures in retry logic under node crashes, and version mismatches during read operations. This was particularly useful in the context of NimbusVault's adaptive replication lifecycle, where chunks dynamically transition between different states.

Benchmarking and Performance Analysis: Designing and validating a benchmarking framework required careful consideration. AI tools assisted in structuring benchmarking scripts, interpreting latency metrics (p50, p95, p99) and throughput trends, and identifying inconsistencies in performance measurements. This

helped ensure that our evaluation accurately reflected system behavior under varying workloads.

Understanding Complex Inter-Service Communication: NimbusVault consists of multiple interacting components, including clients, metadata servers, and storage workers. AI tools helped us reason about gRPC communication patterns, design retry and fallback mechanisms, and handle intermediate system states during replication changes. This was especially useful when implementing transitional states in replication, client-side fallback logic, and distributed coordination between services.

Assistance in Implementing Replication Policies: While the adaptive replication policy was designed by our team, AI tools helped validate logic for promoting and demoting replication levels, suggest edge-case handling to avoid instability, and improve clarity and structure of the implementation.

Code Review and Insight Generation: AI tools acted as an additional layer of review by identifying potential inefficiencies, suggesting improvements in code structure, and offering alternative approaches to implementation. This contributed to improved code quality and better understanding of design trade-offs.

8.3 What Did Not Work Well

Despite the benefits, we encountered several challenges:

Conflicting Suggestions Across Models: Using multiple AI tools sometimes resulted in contradictory implementation suggestions, different assumptions about system behavior, and inconsistent coding styles. In some cases, these conflicts led to unintended changes in stable components, causing regressions and development slowdowns.

Mitigation: We addressed this by freezing inter-service communication interfaces early, ensuring all AI-assisted changes adhered strictly to these interfaces, and avoiding modifications to stable components unless necessary.

Context Loss and Fragmented Understanding: AI tools did not always have full visibility into the system, leading to suggestions that conflicted with existing logic or introduced redundant or incompatible code.

Mitigation: We maintained a centralized project document that tracked implemented components, pending tasks, API contracts and boundaries, and system constraints. This document was used to provide consistent context when interacting with AI tools, reducing inconsistencies.

Overconfidence in Incorrect Outputs: AI tools occasionally produced outputs that appeared correct but contained subtle flaws, especially in complex distributed scenarios.

Mitigation: We treated AI outputs strictly as suggestions. All code was manually reviewed, changes were validated through unit and integration testing, and system behavior was verified under realistic conditions.

8.4 Key Takeaways

Our experience highlights that AI tools are most effective when used as *assistive copilots rather than primary developers*. They significantly improved productivity in debugging, implementation, and analysis, but required careful oversight.

Maintaining control over system design and enforcing strict validation practices allowed us to benefit from AI without compromising correctness. This approach enabled us to accelerate development while preserving the integrity and originality of NimbusVault.

Appendix: Example Prompts

- "Explain possible failure cases in a distributed replication system with dynamic replication changes."
- "Help debug retry logic issues in a distributed gRPC-based system."
- "Suggest improvements for handling state transitions in a distributed storage system."
- "What are edge cases when dynamically changing replication levels in a distributed system?"